

Arbeitsanweisung: Funkintegration mit Chatty-Claudine-Workflow

1. Ziel

Die bestehende Funkverbindung mit dem **nRF24L01+** soll in das Projekt **robot** integriert werden.

Der **vordere Arduino Nano** übernimmt zunächst die Rolle des Senders. Übertragen werden sollen dieselben Telemetriedaten, die aktuell über die serielle Schnittstelle an das Python-Programm gesendet werden.

Zusätzlich soll im Projektordner **robot** ein einfacher Empfänger-Sketch erstellt werden. Der Empfänger dient zunächst nur dazu, die empfangenen Daten über den seriellen Monitor auszugeben.

2. Feste Hardwarevorgaben

Sender: Nano vorne

Signal	Arduino-Nano-Pin
CE	D7
CSN / CS	D8
MOSI	D11
MISO	D12
SCK	D13
VCC	3,3 V
GND	GND

Für den Sender ist folgende Initialisierung zu verwenden:

```
// CE an D7, CSN an D8
RF24 radio(7, 8);
```

Weitere Funkparameter aus dem bereits funktionierenden Testaufbau:

```
const byte adresse[6] = "IGOR1";

radio.setPALevel(RF24_PA_LOW);
radio.setDataRate(RF24_1MBPS);
radio.setChannel(76);
```

Diese Werte dürfen während der ersten Integration nicht ohne Begründung geändert werden.

3. Funktionsumfang des Senders

Der Code für den Sender wird in den vorhandenen Code des **Nano vorne** integriert.

Zu übertragen sind die Daten, die bisher normalerweise über **Serial** an das Python-Programm gesendet werden. Dazu gehören insbesondere die vorhandenen Telemetrieausgaben, beispielsweise:

- Odometrie
- Raddrehzahlen beziehungsweise Radgeschwindigkeiten
- Sollwerte
- Statusmeldungen
- Ultraschallmesswerte
- später gegebenenfalls IMU-Daten

Die bestehende serielle Ausgabe an Python soll zunächst erhalten bleiben. Die Funkübertragung wird parallel ergänzt.

Die Funkintegration darf:

- die bestehende Motorregelung nicht blockieren,
 - den Frame-Scheduler nicht unnötig verzögern,
 - keine langen Wartezeiten verwenden,
 - keine **delay()**-basierte Ablaufsteuerung einführen,
 - die UART-Kommunikation zwischen den beiden Nanos nicht beeinträchtigen.
-

4. Übertragungsprinzip

Für die erste Version sollen möglichst dieselben Textzeilen übertragen werden, die auch an Python ausgegeben werden.

Da ein nRF24L01+-Paket maximal 32 Byte Nutzdaten übertragen kann, müssen längere Telemetriezeilen in mehrere Funkpakete aufgeteilt werden.

Ein Funkpaket soll mindestens folgende Informationen enthalten:

- laufende Nachrichtennummer,
- Teilpaketnummer,
- Anzahl der Teilpakete,
- Nutzdaten.

Der Empfänger setzt die Teilpakete wieder zu einer vollständigen Textzeile zusammen und gibt diese anschließend über **Serial** aus.

Die Paketstruktur muss eindeutig dokumentiert werden.

Die Funkübertragung darf nicht direkt an vielen verschiedenen Stellen des Programms eingebaut werden. Es soll einen zentralen Übergabepunkt für Telemetriedaten geben.

5. Einordnung in den vorhandenen Schichtenaufbau

Der vorhandene Schichtenaufbau des Projekts ist beizubehalten:

```
1_Common  
2_Hardware  
3_Control  
4_Vehicle  
5_System  
Apps
```

Es soll zunächst **keine neue oberste Schicht** eingeführt werden.

Empfohlene Einordnung:

1_Common

Hierhin gehören ausschließlich gemeinsam verwendete Definitionen, zum Beispiel:

- Funkpaketstruktur
- maximale Paketgröße
- Nachrichtentypen
- gemeinsame Konstanten
- Prüfsummen- oder Sequenzdefinitionen

Mögliche Dateien:

```
1_Common/RadioProtocol.h
```

2_Hardware

Hierhin gehört die direkte Ansteuerung des nRF24L01+.

Mögliche Dateien:

```
2_Hardware/Nrf24Radio.h  
2_Hardware/Nrf24Radio.cpp
```

Diese Klasse kapselt ausschließlich:

- Initialisierung des Moduls,
- Schreiben eines Funkpakets,
- Status des Moduls,
- gegebenenfalls einfache Fehlerzähler.

Sie soll keine Odometrie oder Telemetrieinhalte interpretieren.

5_System

Hierhin gehört die Verbindung zwischen der bestehenden Telemetrie und dem Funkmodul.

Mögliche Dateien:

```
5_System/RadioTelemetrySender.h  
5_System/RadioTelemetrySender.cpp
```

Diese Komponente übernimmt:

- vollständige Telemetriezeilen,
- Zerlegung in mehrere Funkpakete,
- Vergabe der Nachrichtennummer,
- Übergabe der Einzelpakete an **Nrf24Radio**.

Apps

Die Initialisierung und zyklische Verwendung wird in die bestehende Front-Anwendung eingebunden.

Die Hauptanwendung soll nur:

- das Funkmodul initialisieren,
- Telemetriedaten an den Radio-Telemetriesender übergeben,
- Fehlerzustände auswerten.

6. Anforderungen an die Integration im Nano vorne

Vor Änderungen ist zu untersuchen, an welcher Stelle die Telemetriedaten aktuell zentral an Python ausgegeben werden.

Bevorzugt soll die vorhandene Telemetriausgabe so erweitert werden, dass dieselbe vollständige Zeile sowohl:

1. an **Serial** und
2. an den Funk-Telemetriesender

übergeben wird.

Doppelter Formatierungscode ist zu vermeiden.

Die Integration muss speichersparend erfolgen. Der Arduino Nano besitzt nur 2 KB SRAM.

Zu beachten:

- keine großen lokalen Zeichenpuffer,
- keine dynamische Speicherverwaltung,
- nach Möglichkeit keine Arduino-**String**-Objekte,
- feste Puffergrößen,
- keine unnötigen Kopien,

- Puffergrenzen konsequent prüfen.

Nach der Integration sind Flash- und SRAM-Verbrauch zu dokumentieren.

7. Einfacher Empfänger

Im Projektordner **robot** soll ein eigener Unterordner für den ersten Empfänger angelegt werden.

Vorschlag:

```
robot/  
└─ funk_empfaenger/  
   └─ funk_empfaenger.ino
```

Für den Empfänger ist zunächst kein Klassenaufbau erforderlich.

Einfache Funktionen sind ausreichend, zum Beispiel:

```
void initRadio();  
bool receivePacket();  
bool processPacket();  
void printCompleteMessage();
```

Aufgaben des Empfängers:

- nRF24L01+ initialisieren,
- Funkpakete empfangen,
- Teilpakete anhand der Nachrichtennummer zusammensetzen,
- unvollständige oder fehlerhafte Nachrichten verwerfen,
- vollständige Telemetriezeilen über **Serial** ausgeben.

Die Ausgabe soll so erfolgen, dass sie später möglichst direkt vom bestehenden Python-Programm verarbeitet werden kann.

Der Empfänger soll zunächst keine Steuerbefehle zurücksenden.

8. Fehlerbehandlung

Mindestens folgende Fehlerfälle sind zu berücksichtigen:

- nRF24L01+ beim Start nicht erkannt,
- einzelnes Teilpaket fehlt,
- falsche Reihenfolge der Teilpakete,
- neue Nachricht beginnt, bevor die alte vollständig ist,
- Nachricht ist länger als der Empfangspuffer,
- Funkübertragung schlägt fehl,

- Paket enthält ungültige Längenangaben.

Fehler dürfen nicht zu Speicherüberläufen oder zum Blockieren des Roboters führen.

Für Diagnosezwecke sollen einfache Zähler vorgesehen werden, zum Beispiel:

- erfolgreich gesendete Pakete,
 - fehlgeschlagene Sendeveruche,
 - empfangene Pakete,
 - verworfene Pakete,
 - unvollständige Nachrichten.
-

9. Chatty-Claudine-Workflow

Phase 1: Analyse durch Chatty

Chatty untersucht zunächst:

- den aktuellen Telemetripfad,
- die beteiligten Dateien,
- den vorhandenen Scheduler,
- die Pinbelegung,
- den verfügbaren Flash- und SRAM-Speicher,
- mögliche Konflikte mit D7, D8 und SPI.

Chatty erstellt danach einen konkreten Änderungsvorschlag mit Dateiliste.

Noch keine bestehenden Projektdateien überschreiben.

Phase 2: Entwurf durch Chatty

Chatty erstellt die neuen beziehungsweise geänderten Dateien zunächst vollständig im Arbeitsbereich **chatty**.

Der Entwurf muss enthalten:

- gemeinsame Funkprotokoll-Definition,
- Hardware-Kapselung für den nRF24L01+,
- Telemetrie-Zerlegung,
- Integration in die Front-Anwendung,
- einfachen Empfänger-Sketch,
- kurze technische Dokumentation.

Phase 3: Kontrolle durch Claudine

Claudine prüft insbesondere:

- Einhaltung des Schichtenaufbaus,
- Pin-Konflikte,
- SRAM-Verbrauch,
- mögliche Blockierungen,

- Paketgrößen,
- Puffergrenzen,
- Reihenfolge und Verlust von Teilpaketen,
- Nebenwirkungen auf Motorregelung, UART und Scheduler,
- Kompilierbarkeit für `nano_atmega328old`.

Claudine soll konkrete Fehler und Verbesserungsvorschläge nennen und nicht nur allgemein bestätigen.

Phase 4: Überarbeitung durch Chatty

Chatty übernimmt die begründeten Korrekturen von Claudine und erstellt eine überarbeitete Endfassung.

Phase 5: Übernahme ins Projekt

Erst nach Abschluss der Kontrolle werden die finalen Dateien in die echte Projektstruktur übernommen.

Bestehende Dateien sind vollständig zu sichern beziehungsweise über Git nachvollziehbar zu ändern.

10. Erwartete Ergebnisse

Am Ende müssen mindestens folgende Ergebnisse vorliegen:

1. Dokumentierte Funkpaketstruktur.
2. Neue Hardware-Kapselung für das nRF24L01+ im Frontprojekt.
3. Zentrale Komponente zum Senden der Telemetriedaten.
4. Integration in den vorhandenen Front-Code.
5. Einfacher Empfänger-Sketch im Projektordner `robot`.
6. Dokumentation aller geänderten und neuen Dateien.
7. Kompilierergebnis für Nano vorne.
8. Flash- und SRAM-Verbrauch vor und nach der Integration.
9. Testbeschreibung.
10. Ergebnis der Claudine-Codeprüfung.

11. Abnahmekriterien

Die Aufgabe gilt als erfolgreich abgeschlossen, wenn:

- der Nano vorne weiterhin normal startet,
- die Kommunikation mit dem hinteren Nano weiterhin funktioniert,
- die Motorregelung unverändert arbeitet,
- die bisherigen seriellen Telemetriedaten weiterhin an Python gesendet werden,
- dieselben Daten zusätzlich per Funk übertragen werden,
- der Empfänger vollständige Telemetriezeilen am seriellen Monitor ausgibt,
- längere Nachrichten korrekt in Teilpakete zerlegt und zusammengesetzt werden,
- verlorene Pakete nicht zu fehlerhaften zusammengesetzten Nachrichten führen,
- keine Speicherüberläufe auftreten,
- der Code für den alten Nano-Bootloader kompiliert wird.

12. Vorgegebene Testreihenfolge

1. Empfänger allein starten und Initialisierung prüfen.
 2. Einfaches festes Testwort senden.
 3. Nachricht mit weniger als 32 Byte übertragen.
 4. Nachricht mit mehr als 32 Byte übertragen.
 5. Mehrere Nachrichten direkt hintereinander übertragen.
 6. Bestehende Telemetriezeilen aus dem Frontprogramm senden.
 7. Gleichzeitige serielle und Funk-Ausgabe prüfen.
 8. Test mit eingeschalteter Motorregelung im Stand.
 9. Test während einer langsamen Fahrt.
 10. Paketfehler- und Speicherzähler auswerten.
 11. Abschließend Flash- und SRAM-Verbrauch dokumentieren.
-

13. Wichtige Arbeitsregel

Keine vorschnelle Komplettänderung des Frontprogramms.

Zuerst den bestehenden Datenfluss analysieren, danach die kleinste saubere Erweiterung entwerfen. Die vorhandene Architektur ist wichtiger als eine schnelle Einzellösung.